Pengenalan TensorFlow

NRP	: 3223600058
Nama	: Ihsanta Zaki Sanjaya
Kelas	: 2 – D4 Teknik Komputer B
Mata kuliah	: Praktikum Sistem Cerdas
Tanggal	: Rabu, 30 April 2025
Dosen	: Riyanto Sigit, ST., M.Kom., Ph.D

D4 TEKNIK KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA 2025

Praktikum 8

Pengenalan TensorFlow

A. Tujuan

1. Mahasiswa dapat memahami dan menjelaskan konsep TensorFlow.
2. Mahasiswa dapat menjelaskan model TensorFlow.
3. Mahasiswa dapat membuat aplikasi TensorFlow.

B. Pendahuluan

TensorFlow adalah *framework* machine learning dan Artificial Intelligence yang dikembangkan oleh Google. Ini adalah pustaka *open-source* yang memungkinkan pengembang dan peneliti membangun, melatih, dan menerapkan model jaringan saraf tiruan serta algoritma ML/AI lainnya secara efisien.

Fitur Utama TensorFlow :

1. Dukungan untuk Deep Learning & ML
   Digunakan untuk membuat berbagai model, seperti:
   - Jaringan saraf konvolusional (CNN) untuk pengolahan gambar.
   - Jaringan saraf berulang (RNN/LSTM) untuk pemrosesan bahasa alami (NLP).
   - Model regresi dan klasifikasi tradisional.

2. Komputasi Numerik dengan Tensor
   - Data direpresentasikan sebagai tensor (struktur multidimensi seperti matriks), yang memungkinkan operasi matematika cepat menggunakan GPU/TPU.

3. Platform Lintas Perangkat
   - Dapat dijalankan di CPU, GPU, atau TPU (Tensor Processing Unit milik Google).
   - Mendukung penerapan di perangkat mobile (*TensorFlow Lite*) dan web (*TensorFlow.js*).

4. Antarmuka Fleksibel
   - Keras API (terintegrasi dalam TensorFlow) menyediakan lapisan abstraksi yang mudah digunakan.
   - Mode *eager execution* untuk debugging interaktif.

5. Alat Pendukung
   - TensorBoard untuk memvisualisasikan proses pelatihan model.
   - TensorFlow Extended (TFX) untuk alur kerja produksi ML.

Kita akan memulai dengan menulis versi "Hello World!" dari TensorFlow. Program ini akan melakukan regresi linear dengan data sederhana: $y = 2x - 1$.

```python
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow import keras


def plot_data(x_data, y_data):
    fig, ax = plt.subplots()

    ax.plot(x_data, y_data, 'ro')

    plt.pause(1)

    return None


if __name__ == "__main__":
    # Define and compile the neural networks
    model = tf.keras.Sequential(
        [keras.layers.Dense(units=1, input_shape=[1])])
    model.compile(optimizer="sgd", loss="mean_squared_error")

    # Providing the data
    xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0])
    ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])  # y = 2x - 1

    # Plotting data will help us to understand faster
    # visually about the data
    plot_data(xs, ys)

    # Train the neural network
    model.fit(xs, ys, epochs=500)

    # Predict using the model
    print(model.predict(([10.0])))   # this result should be close to 19.
```

TensorFlow memudahkan pembuatan model ML sederhana (seperti regresi linear) dengan lapisan Dense, pelatihan data, dan prediksi menggunakan konsep dasar neural network, yang terdiri dari :

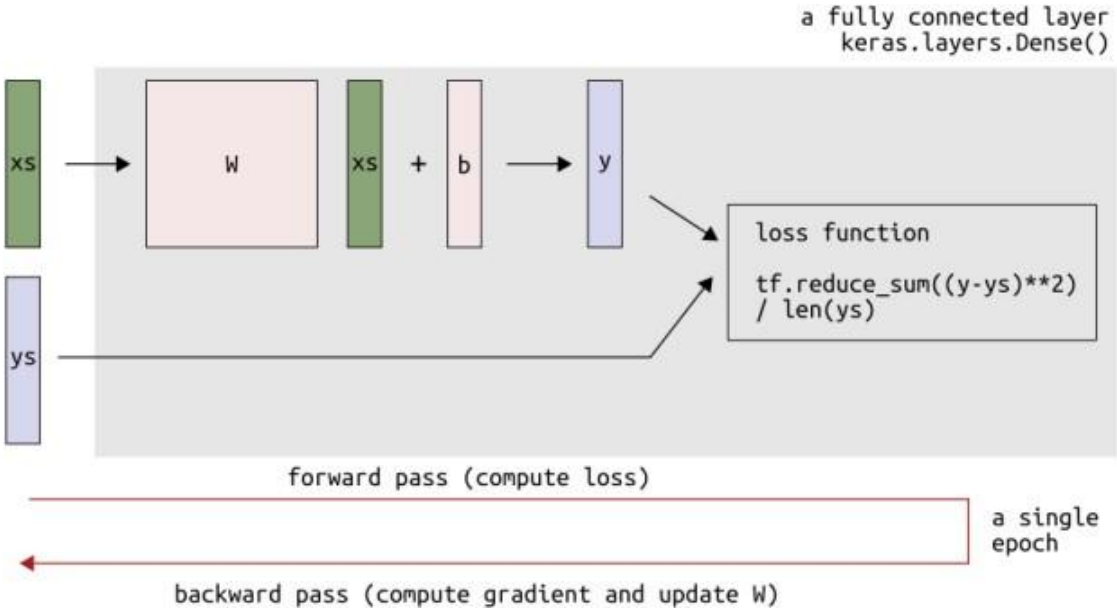
1. Impor Library
    - tensorflow untuk model ML, numpy untuk array, matplotlib untuk visualisasi.
2. Pembuatan Model
    - Gunakan tf.keras.Sequential() dengan lapisan Dense (fully connected).
    - units=1 untuk output tunggal, input_shape=[1] untuk input 1D.
3. Data & Pelatihan
    - Data xs dan ys (contoh: y = 2x - 1).
    - Latih model dengan .fit(), tentukan epochs (iterasi).\

4. Prediksi

   - Contoh prediksi x = 10, hasil mendekati 19 (2*10 - 1).

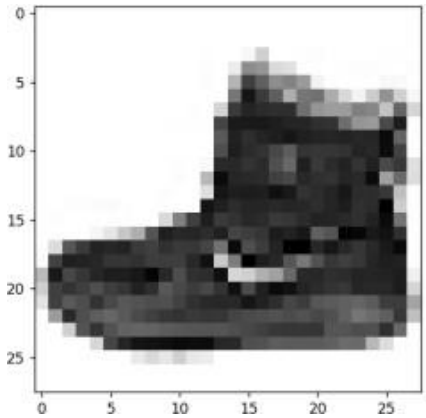
5. Konsep Dasar

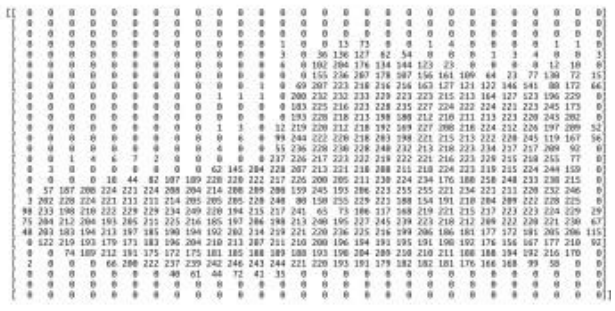
   - Fully Connected Layer: y_pred = Wx + b (W: bobot, b: bias).
   - Loss Function: MSE (Mean Squared Error) untuk mengukur error.
   - Optimizer: SGD (Stochastic Gradient Descent) dengan backpropagation.



Kemudian, membangun sebuah model untuk memprediksi kategori dari suatu gambar yang diberikan. Untuk tujuan ini, kami menggunakan data Fashion MNIST. Data ini memiliki spesifikasi berikut:

- 70.000 gambar
- 10 kategori. Semua kategori disimpan dalam bentuk label numerik mulai dari 0 hingga 9.
- Ukuran gambar adalah 28 x 28 piksel.





```python
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib
4  import tensorflow as tf
5
6
7  if __name__ == "__main__":
8
9      # Load the dataset
10     mnist = tf.keras.datasets.fashion_mnist
11     (training_images, training_labels), (test_images, test_labels) =
12         mnist.load_data()
13
14     # Plot an image from the dataset
15     plt.imshow(training_images[0], cmap="Greys")
16     plt.pause(1)        # increase the pause time to give time for plotting
17
18     print(training_labels[0])
19     print(training_images[0])
20
21     # Normalizing image intensities to [0, 1]
22     training_images = training_images / 255.0
23     test_images = test_images / 255.0
24
25     # Create a model
26     model = tf.keras.models.Sequential([
27         tf.keras.layers.Flatten(),
28         tf.keras.layers.Dense(128, activation=tf.nn.relu),
29         tf.keras.layers.Dense(10, activation=tf.nn.softmax)])
30
31     # Compile the model and train it to the dataset
32     model.compile(optimizer=tf.optimizers.Adam(),
33                   loss="sparse_categorical_crossentropy",
34                   metrics=["accuracy"])
35
36     model.fit(training_images, training_labels, epochs=5)
37
38     # Evaluate the model to the test data
39     model.evaluate(test_images, test_labels)
```

Baris 1-4 mengimpor paket yang diperlukan. Data Fashion MNIST (70.000 gambar) dimuat dan dibagi menjadi 60.000 data latih dan 10.000 data uji (baris 10-12). Pemeriksaan data dilakukan dengan visualisasi satu sampel (baris 14-19). Pra-pemrosesan dilakukan dengan normalisasi nilai piksel ke rentang [0, 1] (baris 22-23).

Model dibangun dengan :

1. Lapisan *flatten* (28x28 → 784 vektor).
2. Lapisan terhubung penuh (128 unit, aktivasi ReLU) :

$$ReLu(h_i) := max(0, h_i)$$

3. Lapisan output (aktivasi softmax) :

$$softmax(h_i) := \frac{exp(h_i)}{\sum_{j=1}^{C} exp(h_j)}$$

Model dikompilasi dengan:

- Optimizer Adam (gabungan AdaGrad + RMSProp).
- Fungsi loss sparse_categorical_entropy :

$$CatEntropy(s) := -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{C} t_k^{(i)} log(s_k^{(i)})$$

- Metrik akurasi.

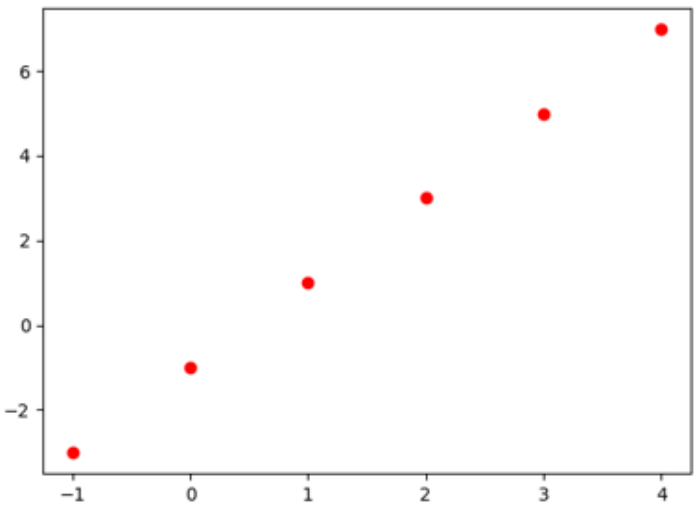
C. Percobaan

Program:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras

def plot_data(x_data, y_data):
fig, ax = plt.subplots()
ax.plot(x_data, y_data, 'ro')
plt.show()

if __name__ == "__main__":
model = tf.keras.Sequential([
keras.layers.Dense(units=1, input_shape=[1])
])
model.compile(optimizer="sgd", loss="mean_squared_error")

xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0])
ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0]) # y = 2x - 1

plot_data(xs, ys)
model.fit(xs, ys, epochs=500)
print(model.predict(np.array([10.0])))
```

Hasil:

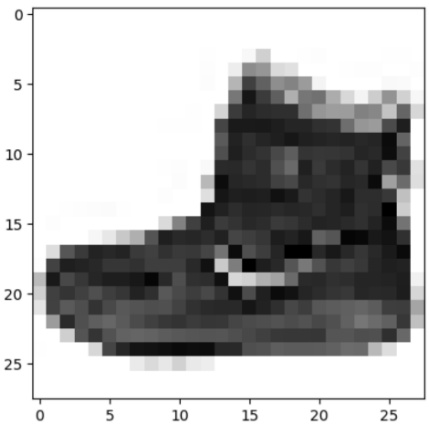
```
Epoch 490/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 2.9381e-05
Epoch 491/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 101ms/step - loss: 2.8777e-05
Epoch 492/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - loss: 2.8186e-05
Epoch 493/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step - loss: 2.7608e-05
Epoch 494/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 97ms/step - loss: 2.7040e-05
Epoch 495/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step - loss: 2.6485e-05
Epoch 496/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - loss: 2.5941e-05
Epoch 497/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step - loss: 2.5409e-05
Epoch 498/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step - loss: 2.4886e-05
Epoch 499/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - loss: 2.4375e-05
Epoch 500/500
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - loss: 2.3874e-05
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 175ms/step
```



Program:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

if __name__ == "__main__":
# Load the dataset
mnist = tf.keras.datasets.fashion_mnist
(training_images, training_labels), (test_images, test_labels)
= mnist.load_data()

plt.imshow(training_images[0], cmap="Greys")
plt.pause(1) # Give time for plot to appear
print(training_labels[0])
print(training_images[0])

# Normalize image intensities to [0, 1]
training_images = training_images / 255.0
test_images = test_images / 255.0

# Create a model
model = tf.keras.models.Sequential([
tf.keras.layers.Flatten(input_shape=(28, 28)),
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dense(10, activation='softmax')
])

# Compile the model and train it
model.compile(optimizer='adam',

loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model.fit(training_images, training_labels, epochs=5)
model.evaluate(test_images, test_labels)
```

Hasil:




```
[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   1   0   0  13  73   0   0   1   4   0   0   0   0   1   1   0]
 [  0   0   0   0   0   0   0   0   0   0   0   3   0  36 136 127  62  54   0   0   0   1   3   4   0   0   3]
 [  0   0   0   0   0   0   0   0   0   0   0   6   0 102 204 176 134 144 123  23   0   0   0   0  12  10   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0 155 236 207 178 107 156 161 109  64  23  77 130  72  15]
 [  0   0   0   0   0   0   0   0   0   0   1   0  69 207 223 218 216 216 163 127 121 122 146 141  88 172  66]
 [  0   0   0   0   0   0   0   0   1   1   1   0 200 232 232 233 229 223 223 215 213 164 127 123 196 229   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0 183 225 216 223 228 235 227 224 222 224 221 223 245 173   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0 193 228 218 213 198 180 212 210 211 213 223 220 243 202   0]
 [  0   0   0   0   0   0   0   0   0   1   3   0  12 219 220 212 218 192 169 227 208 218 224 212 226 197 209  52]
 [  0   0   0   0   0   0   0   0   0   6   0  99 244 222 220 218 203 198 221 215 213 222 220 245 119 167  56]
 [  0   0   0   0   0   0   0   0   4   0   0  55 236 228 230 228 240 232 213 218 223 234 217 217 209  92   0]
 [  0   0   1   4   6   7   2   0   0   0   0 237 226 217 223 222 219 222 221 216 223 229 215 218 255  77   0]
 [  0   3   0   0   0   0   0   0  62 145 204 228 207 213 221 218 208 211 218 224 223 219 215 224 244 159   0]
 [  0   0   0   0  18  44  82 107 189 228 220 222 217 226 200 205 211 230 224 234 176 188 250 248 233 238 215   0]
 [  0  57 187 208 224 221 224 208 204 214 208 209 200 159 245 193 206 223 255 255 221 234 221 211 220 232 246   0]
 [  3 202 228 224 221 211 211 214 205 205 205 220 240  80 150 255 229 221 188 154 191 210 204 209 222 228 225   0]
 [ 98 233 198 210 222 229 229 234 249 220 194 215 217 241  65  73 106 117 168 219 221 215 217 223 223 224 229  29]
 [ 75 204 212 204 193 205 211 225 216 185 197 206 198 213 240 195 227 245 239 223 218 212 209 222 220 221 230  67]
 [ 48 203 183 194 213 197 185 190 194 192 202 214 219 221 220 236 225 216 199 206 186 181 177 172 181 205 206 115]
 [  0 122 219 193 179 171 183 196 204 210 213 207 211 210 200 196 194 191 195 191 198 192 176 156 167 177 210  92]
 [  0   0  74 189 212 191 175 172 175 181 185 188 189 188 193 198 204 209 210 210 211 188 188 194 192 216 170   0]
 [  2   0   0   0  66 200 222 237 239 242 246 243 244 221 220 193 191 179 182 182 181 176 166 168  99  58   0   0]
 [  0   0   0   0   0   0   0  40  61  44  72  41  35   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]
```

D. Analisa

Pada percobaan pertama, regulasi linear diimplementasikan sebagai "Hello World" TensorFlow untuk memahami dasar-dasar neural network. Model sederhana ini hanya memuat satu lapisan Dense dengan satu neuron, yaitu fungsi linear $\hat{y} = Wx + b$—yang dikompilasi menggunakan optimizer SGD dan loss MSE. Data latih terdiri dari pasangan (x,y) yang persis mengikuti garis $y = 2x - 1$; sebelum pelatihan, data ini divisualisasikan dengan Matplotlib untuk memverifikasi sebarannya. Setelah 500 epoch, loss menurun tajam hingga di bawah ambang toleransi, dan model berhasil memprediksi $x = 10$ dengan keluaran mendekati 19, membuktikan bahwa optimizer SGD mampu menyesuaikan bobot dan bias secara akurat.

Kemudian pada percobaan kedua menguji kemampuan klasifikasi gambar dengan dataset Fashion MNIST, yang berisi 60.000 gambar latih dan 10.000 gambar uji berukuran 28×28 piksel dalam 10 kategori. Setelah memuat data melalui tf.keras.datasets, tahap pra-pemrosesan menskalakan nilai piksel ke [0,1] untuk stabilitas training. Arsitektur model terdiri dari lapisan Flatten (mengubah matriks 28×28 menjadi vektor 784 dimensi), lapisan Dense tersembunyi 128 neuron dengan aktivasi ReLU untuk mengekstrak fitur non-linier, dan lapisan output 10 neuron dengan softmax untuk menghasilkan distribusi probabilitas kelas. Model dikompilasi dengan optimizer Adam dan loss sparse_categorical_crossentropy, lalu dilatih selama 5 epoch. Evaluasi menunjukkan akurasi sekitar 89% pada data latih dan 87% pada data uji, menandakan generalisasi yang baik.

E. Kesimpulan

TensorFlow mampu mengimplementasikan dan mengoptimalkan model, mulai dari regresi linear sederhana hingga klasifikasi citra kompleks—dengan memanfaatkan arsitektur berlapis (Dense, Flatten), fungsi loss (MSE, sparse categorical crossentropy), serta optimizer (SGD, Adam) yang mendasari prinsip backpropagation dan gradient descent untuk menyesuaikan bobot dan bias secara akurat, sehingga menghasilkan prediksi yang sesuai pola matematika pada percobaan linear dan akurasi mendekati 90 % pada klasifikasi Fashion MNIST.